

COMPUTER FUNDAMENTALS

Xin Liu/Computer center

Chapter3 Algorithms

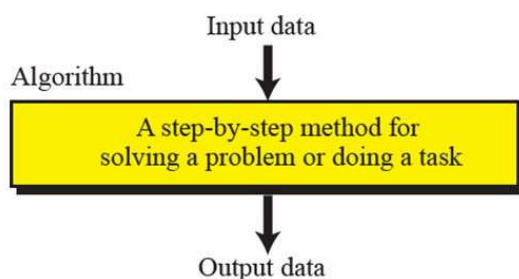
Agenda

- What is an Algorithm?
- Use of Algorithms
- Characteristic of an Algorithm
- How to Express an Algorithm?
- Algorithm Analysis
- Algorithm Complexity
- Pros and Cons of an Algorithm
- Algorithm vs Programming

What is an Algorithm?

Informal definition

- **Algorithm:** a step-by-step method for solving a problem or doing a task.



A more formal definition

- **Algorithm:** An ordered set of unambiguous steps that produces a result and terminates in a finite time.
- **Characteristics:**
 - Input
 - Output
 - Definiteness
 - Finiteness
 - Effectiveness

What is an Algorithm?



She is Sara, and she wants to prepare milk tea. To do so, she is following a series of steps.

What is an Algorithm?

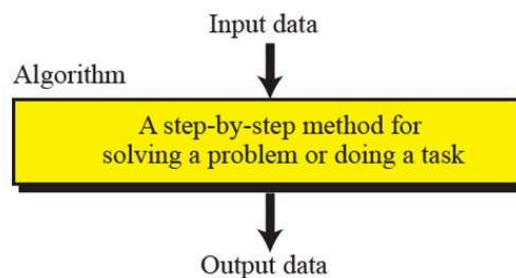
- **STEP 1**
 - Take a pot, fill it with water, and place it on the gas stove.
- **STEP 2**
 - After the water has come to a boil, add the tea leaves and sugar.
- **STEP 3**
 - Allow for full leaf expansion and milking after that.
- **STEP 4**
 - Then wait for the tea to come to a boil.
- **STEP 5**
 - Turn off the gas after it has finished cooking.
- **STEP 6**
 - You' re all set with your tea now.



What is an Algorithm?

The word Algorithm means " A set of finite rules or instructions to be followed in calculations or other problem-solving operations "

Or " A procedure for solving a mathematical problem in a finite number of steps that frequently involves recursive operations".



Use of the Algorithms(1)

1. Computer Science:

–Algorithms form the basis of computer programming and are used to solve problems ranging from simple sorting and searching to complex tasks such as artificial intelligence and machine learning.

2. Mathematics:

–Algorithms are used to solve mathematical problems, such as finding the optimal solution to a system of linear equations or finding the shortest path in a graph.

3. Operations Research:

–Algorithms are used to optimize and make decisions in fields such as transportation, logistics, and resource allocation.

Use of the Algorithms(2)

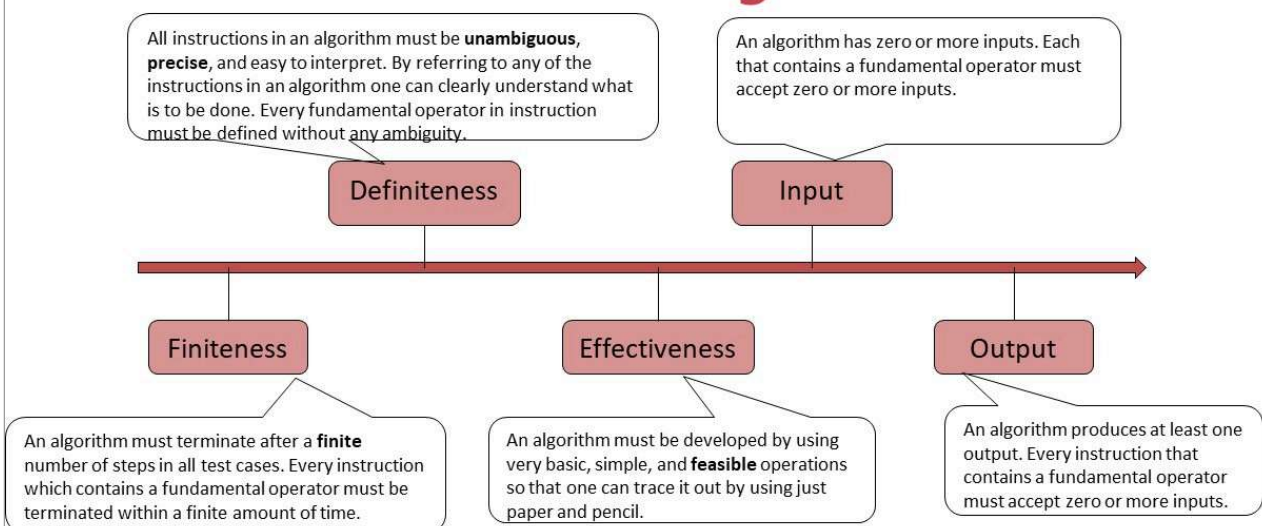
4. Artificial Intelligence:

–Algorithms are the foundation of artificial intelligence and machine learning, and are used to develop intelligent systems that can perform tasks such as image recognition, natural language processing, and decision-making.

5. Data Science:

–Algorithms are used to analyze, process, and extract insights from large amounts of data in fields such as marketing, finance, and healthcare.

Characteristics of an Algorithm



Characteristics of an Algorithm

1. Finiteness:

- ✓ An algorithm must terminate after a finite number of steps in all test cases. Every instruction which contains a fundamental operator must be terminated within a finite amount of time.

2. Definiteness:

- ✓ All instructions in an algorithm must be unambiguous, precise, and easy to interpret. By referring to any of the instructions in an algorithm one can clearly understand what is to be done. Every fundamental operator in instruction must be defined without any ambiguity.

3. Effectiveness:

- ✓ An algorithm must be developed by using very basic, simple, and feasible operations so that one can trace it out by using just paper and pencil.

Characteristics of an Algorithm

4. Input:

- ✓ An algorithm has zero or more inputs. Each that contains a fundamental operator must accept zero or more inputs.

5. Output:

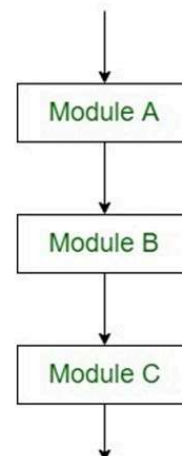
- ✓ An algorithm produces at least one output. Every instruction that contains a fundamental operator must accept zero or more inputs.

Basic Operations

- **Arithmetic Operations**
 - +, -, *, /, **mod(remainder)**, ^
- **Relational Operations:**
 - < less than
 - > greater than
 - <= less than or equal to
 - >= greater than or equal to
 - = equality (equal to)
 - != or <> inequality (not equal to)
- **Logical Operations**
 - && And,
 - || or,
 - ! not
- **Data Transmission**
 - Input, output
- **Assignment**
 - =, ←

Three types of control structures

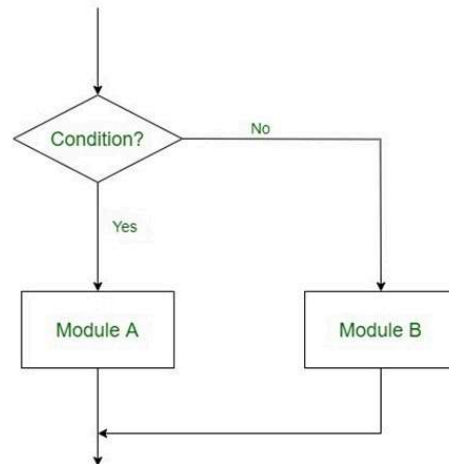
- **Sequential Logic (Sequential Flow)**
 - Sequential logic as the name suggests follows a serial or sequential flow in which the flow depends on the series of instructions given to the computer. Unless new instructions are given, the modules are executed in the obvious sequence. The sequences may be given, by means of numbered steps explicitly. Also, implicitly follows the **order** in which modules are written. Most of the processing, even some complex problems, will generally follow this elementary flow pattern.



Three types of control structures

– Selection Logic (Conditional Flow)

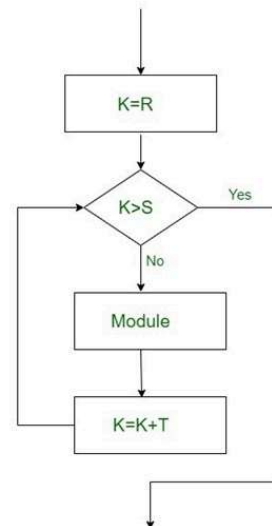
```
If (condition A), then:  
    [Module A]  
Else if (condition B), then:  
    [Module B]  
    ..  
    ..  
Else if (condition N), then:  
    [Module N]  
[End If structure]
```



Three types of control structures

– Iteration Logic (Repetitive Flow)

- The Iteration logic employs a loop which involves a repeat statement followed by a module known as the body of a loop.



How to Express an Algorithm?

1. Natural Language:

- Here we express the Algorithm in the natural English language(or other languages). It is too hard to understand the algorithm from it.

2. Flowchart:

- Here we express the Algorithm by making a graphical/pictorial representation of it. It is easier to understand than Natural Language.

3. Pseudocode:

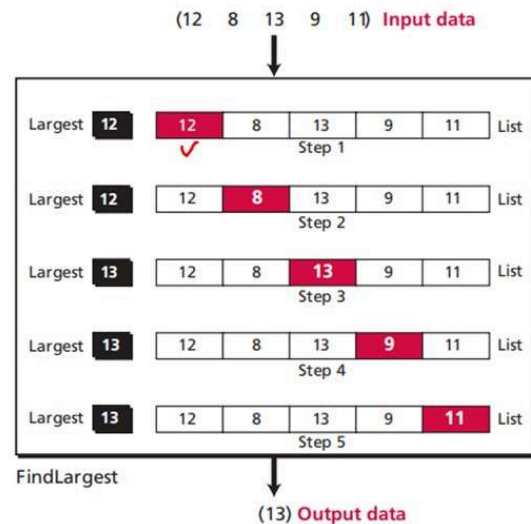
- Here we express the Algorithm in the form of annotations and informative text written in plain English which is very much similar to the real code but as it has no syntax like any of the programming languages, it can't be compiled or interpreted by the computer. It is the best way to express an algorithm because it can be understood by even a layman with some school-level knowledge.

Natural Language:

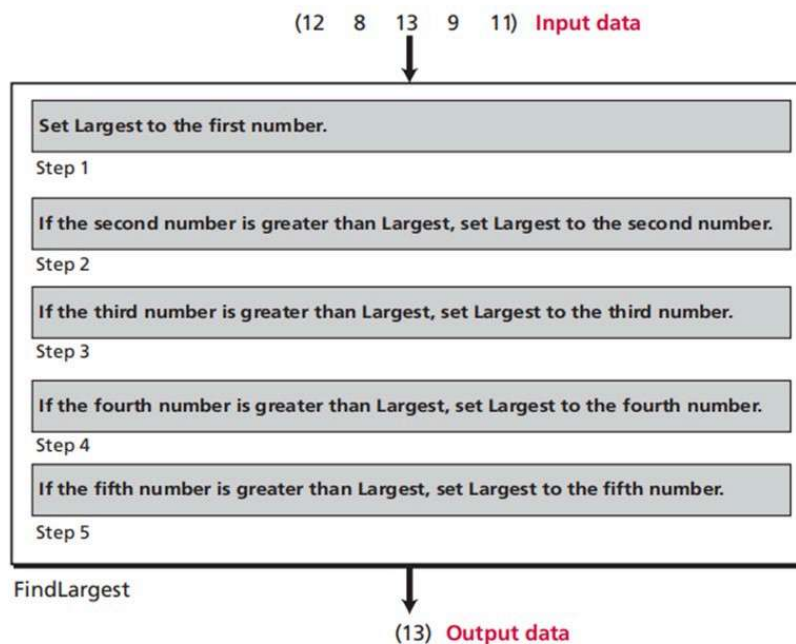
- **AbsoluteValue:** Calculates the absolute value of a number.
 - Step 1
 - Input an integer number X;
 - Step 2
 - If the number X is greater than or equal to 0, output is X;
 - Step 3
 - Otherwise, output is $-X$;

Natural Language

- **FindLargest:** Finding the largest integer among five integers.



Natural Language



Pseudocode

- E.g. How to calculate sum of the two integers?

```
begin
  input a
  input b
  sum=a+b
  output sum
end
```

Pseudocode

- E.g. Input score from keyboard, test if it is larger than or equal to 60. If yes, grade is pass, then, output "pass"; otherwise, grade is failed, output "failed".

```
begin
  input score
  if score>=60 then
    output pass
  else
    output failed
end
```

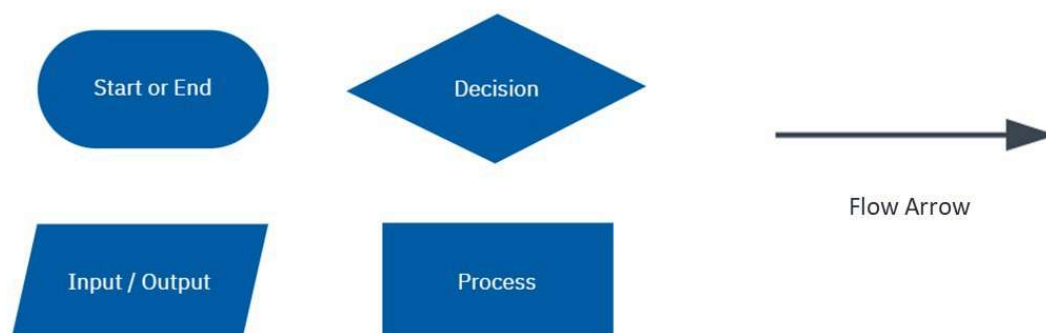
Pseudocode

- E.g. How to calculate $1+2+3+\dots+n=?$

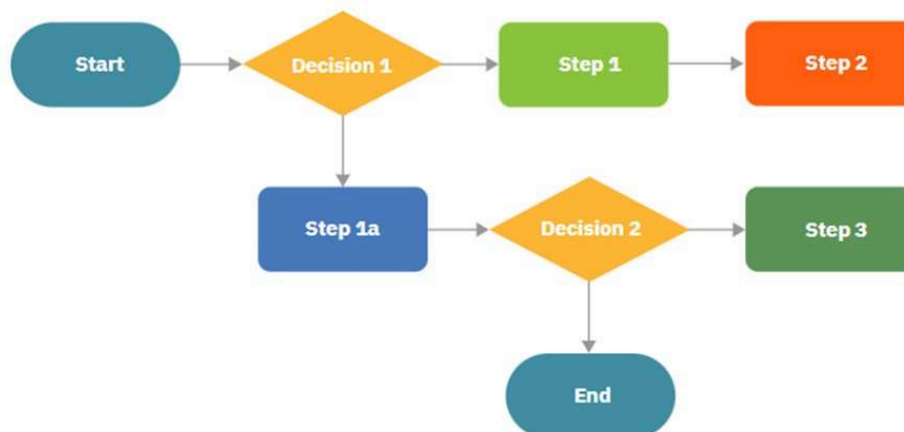
```
begin
  input n
  i = 1
  sum=0
  while i <= n
    sum= sum + i
    i = i+1
  output sum
end
```

Flowchart

- A flowchart is a diagram that depicts a process, system or computer algorithm.
- Flowcharts, sometimes spelled as flow charts, use rectangles, ovals, diamonds and potentially numerous other shapes to define the type of step, along with connecting arrows to define flow and sequence.

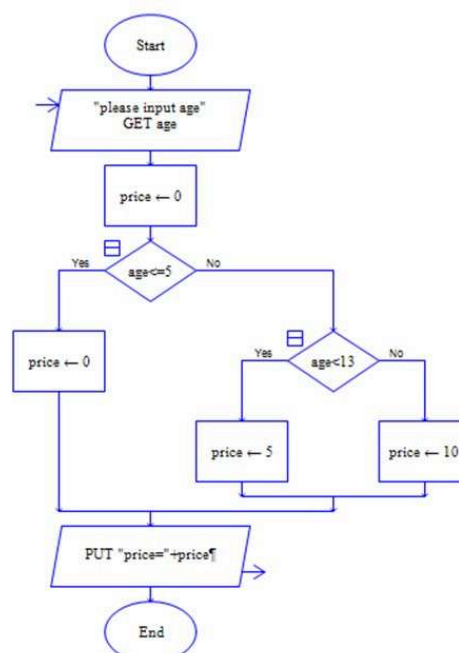


Flowchart



Flowchart

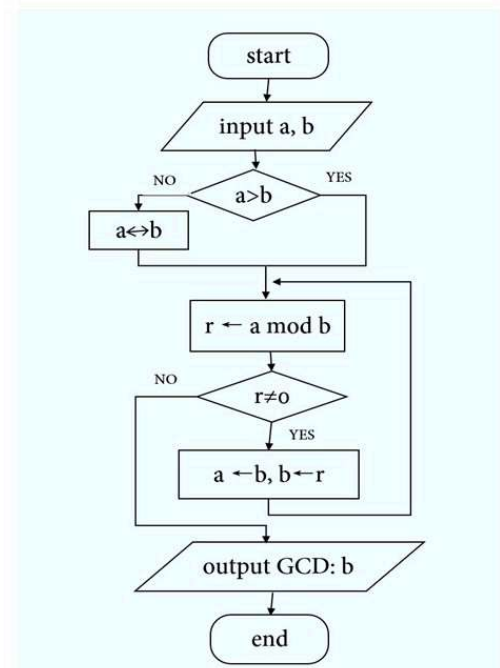
```
begin
  input age
  price=0
  if age<=5 then
    price=0
  else if age<13 then
    price=5
  else
    price=10
  output price
end
```



Flowchart

- **The Euclidean Algorithm**

- **Or Euclid's algorithm, is an efficient method for computing the greatest common divisor (GCD) of two integers.**
 - If we subtract a smaller number from a larger one (we reduce a larger number), GCD doesn't change. So if we keep subtracting repeatedly the larger of two, we end up with GCD.
 - Now instead of subtraction, if we divide the larger number, the algorithm stops when we find the remainder 0.



Design and Analysis of Algorithms

Algorithm vs. Program

Topics

- Introduction
- Algorithm vs. Program
- Summary

Introduction

- An **algorithm** is a step-by-step procedure to solve a specific problem.
- A **program** is also a step-by-step procedure to solve a specific problem.

So, what's the difference?



Topics

- Introduction
- Algorithm vs. Program
- Summary

Algorithm vs. Program

- 1. An algorithm is an **abstract concept**. Whereas, a program is a **concrete implementation** of the algorithm.

Algorithm

1. Start
2. Initialize a variable 'result' to 1.
3. Read the input number 'n'.
4. Repeat the following until 'n' becomes 0;
 - a) Multiply 'result' by 'n'.
 - b) Decrement 'n' by 1.
5. Print the value of 'result' as the factorial.
6. Stop.

Program

```
int fact(int n)
{
    int result = 1;
    while(n>0){
        result *= n;
        n--;
    }
    return result;
}
```

Algorithm vs. Program

- 2. An algorithm can be written in **any language**, while a program must be written using a **programming language only**.

Algorithm

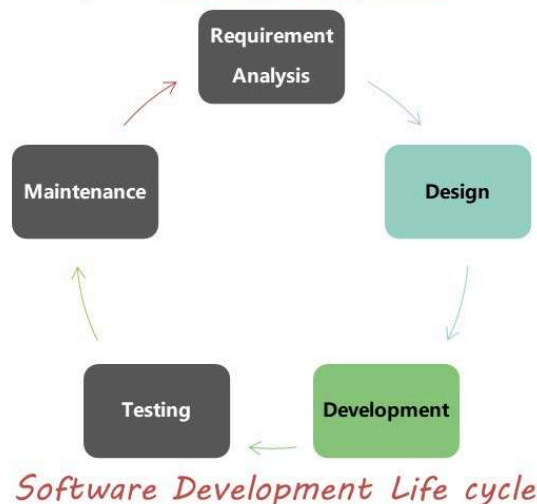
1. Start
2. Initialize a variable 'result' to 1.
3. Read the input number 'n'.
4. Repeat the following until 'n' becomes 0;
 - a) Multiply 'result' by 'n'.
 - b) Decrement 'n' by 1.
5. Print the value of 'result' as the factorial.
6. Stop.

Program

```
int fact(int n)
{
    int result = 1;
    while(n>0){
        result *= n;
        n--;
    }
    return result;
}
```

Algorithm vs. Program

- 3. An algorithm is developed during the **design phase**, and a program is developed during the **development phase**.



Algorithm vs. Program

- 4. An algorithm does not depend on the **hardware** and **operating system** while a program depends upon them.



Algorithm vs. Program

- 5. An algorithm is always **analyzed**, while a program is **tested**.



Topics

- **Introduction**
- **Algorithm vs. Program**
- **Summary**

Summary

- | | |
|--|--|
| <ul style="list-style-type: none">• Algorithm<ul style="list-style-type: none">– 1. It is an abstract concept.– 2. Can be written in any language.– 3. Developed during design phase.– 4. Independent of the hardware and operating system.– 5. It is analyzed. | <ul style="list-style-type: none">• Program<ul style="list-style-type: none">– 1. It is concrete implementation.– 2. Can be written using a programming language.– 3. Developed during development phase.– 4. Depends on the hardware and operating system.– 5. It is tested. |
|--|--|

Design and Analysis of Algorithms

Significance of Algorithms

Significance of Algorithms

A bad algorithm can lead to **longer execution time**.

Bad Algorithm

- 1. Start.
- 2. Initialize the variable sum to 0.
- 3. Enter the loop that iterates from 1 to 10^{12} .
- 4. Add the current iteration value to sum.
- 5. Repeat steps 4 and 5.
- 6. Return the value of sum.
- 7. Stop

Equivalent Program

```
Long int calculate_sum() {  
    long int sum=0, i;  
    for (i=1; i<=1000 000 000 000;i++){  
        sum += i;  
    }  
    return sum;  
}
```

Number of instructions = $2 + 1 + 1000\ 000\ 000\ 000 \times (1+1+1) = 3000\ 000\ 000\ 004$

Significance of Algorithms

A bad algorithm can lead to **longer execution time**.

Number of instructions = $2 + 1 + 1000000000000 \times (1+1+1) = 3,000,000,000,004$

Assume that a computer takes 1 second to execute y instructions, and $y = 100000000$.

Time to run y instructions = 1 second

Time to run 1 instruction = $1/y$ seconds

Time to run x instructions = x/y seconds

Time to run 3000000000004 instructions = $\frac{3,000,000,000,004}{100,000,000} \approx 30,000$
seconds ≈ 500 minutes ≈ 8 hours

Significance of Algorithms

A bad algorithm can lead to **longer execution time**.

Bad Algorithm

- 1. Start.
- 2. Initialize the variable sum to 0.
- 3. Enter the loop that iterates from 1 to 10^{12} .
- 4. Add the current iteration value to sum.
- 5. Repeat steps 4 and 5.
- 6. Return the value of sum.
- 7. Stop

Equivalent Program

```
Long int calculate_sum() {  
    long int sum=0, i;  
    for (i=1; i<=1000000000000;i++){  
        sum += i;  
    }  
    return sum;  
}
```

➤ *Total time ≈ 8 hours*

Significance of Algorithms

A good algorithm can lead to **shorter execution time**.

Good Algorithm

- 1. Start.
- 2. Initialize n to 10^{12}
- 3. Calculate the sum using the formula
$$\text{sum} = \frac{n(n+1)}{2}$$
- 4. Output the value of sum.
- 5. Stop.

$$\begin{aligned}\text{Number of instructions} &= 2 + 4 + 1 \\ &= 7\end{aligned}$$

Equivalent Program

```
Long int calculate_sum() {  
    long int n = 1000000000000;  
    long int sum;  
  
    sum = (n * (n + 1)) / 2;  
    return sum;  
}
```

Significance of Algorithms

A good algorithm can lead to **shorter execution time**.

$$\begin{aligned}\text{Number of instructions} &= 2 + 4 + 1 \\ &= 7\end{aligned}$$

Assume that a computer takes 1 second to execute y instructions, and $y = 100000000$.

Time to run y instructions = 1 second

Time to run 1 instruction = $1/y$ seconds

Time to run x instructions = x/y seconds

$$\text{Time to run 7 instructions} = \frac{7}{100000000} = 0.00000007 \text{ seconds}$$

Significance of Algorithms

A good algorithm can lead to **shorter execution time**.

Good Algorithm

- 1. Start.
- 2. Initialize n to 10^{12}
- 3. Calculate the sum using the formula
$$\text{sum} = \frac{n(n+1)}{2}$$
- 4. Output the value of sum.
- 5. Stop.

Equivalent Program

```
Long int calculate_sum() {  
    long int n = 1000000000000;  
    long int sum;  
  
    sum = (n * (n + 1)) / 2;  
    return sum;  
}
```

➤ *Total time = 0.00000007 seconds*

E.g. 

You are required to both draw a **flow chart** and write down its equivalent **pseudocode** for each of the following three tasks:

- (1) Input an integer number n from the keyboard, test if it is larger than or equal to 80. If yes, then output "A"; otherwise, Output "B".
- (2) First input an integer n, and then n integer numbers to an array grade[] from the keyboard. Output the average of all these n numbers in the array.
- sum/n
- (3) Output **all indices** corresponding to the minimal value of the numbers stored in the array created in (2). For example, if the array is [77, 56, 81, 93, 61, 77, 56, 90], then output indices 2, 7 (corresponding to the minimum 56). *1, min 2, compare*

```

Begin
  input n
  sum = 0
  for i = 1 to n do
    input grade[i]
    sum = sum + grade[i]
  end for
  average = sum/n
  output average
end

```

```

Begin
  min = grade[1]
  for i=2 to n
    if grade[i]<min then
      min = grade[i]
    end for

    for j=1 to n do
      if grade[j] = min then
        output j
      end for
    end for
  end
end

```

Sequential Search

data=70

1	2	3	4	5	6	7	8	9	10
5	9	17	23	25	<u>45</u>	59	63	71	89

Pseudocode:

```

begin
  input a[1], a[2], a[3], ..., a[n]
  input key
  i = 1
  While i<=n and a[i]<>key
    i=i+1
  if i>n then
    output "not found"
  else
    output "found it"
  end
end

```

Binary Search

Sorted

0 1 2 3 4 5 6 7 8 9

5 9 17 23 25 45 59 63 71 89

Case 1: $\text{num} = \text{arr}[\text{mid}]$;
 Case 2: $\text{num} < \text{arr}[\text{mid}]$;
 $\text{right} = \text{mid} - 1$;
 Case 3: $\text{num} > \text{arr}[\text{mid}]$;
 $\text{left} = \text{mid} + 1$;
 $\text{mid} = (\text{left} + \text{right}) / 2$

If $\text{num} = 59$

L	R	Mid
0	9	4
5	9	7
5	6	5
6	6	6

If $\text{num} = 60$

L	R	Mid
0	9	4
5	9	7
5	6	5
6	6	6
7	6	6

$L > R$ ✗

Pseudocode

- function `binarySearch(arr, num)`:
- `left = 0`
- `right = length(arr) - 1`
- while `left <= right`:
- `mid = left + (right - left) / 2` // $\text{mid} = (\text{left} + \text{right}) / 2$
- if `arr[mid] == num`:
- return `mid`
- else if `arr[mid] < num`:
- `left = mid + 1`
- else:
- `right = mid - 1`
- return `-1` // not found

```

1
2 int binary_search(int arr[], int num,int len)
3 {
4     int left = 0;
5     int right = len - 1;
6     int mid = 0;
7     while (left <= right)
8     {
9         mid = left + (right - left) / 2;
10        if (arr[mid] > num)
11        {
12            right = mid - 1;
13        }
14        else if (arr[mid] < num)
15        {
16            left = mid + 1;
17        }
18        else ==
19        {
20            return mid;
21        }
22    }
23
24    return -1; // L > R
25
26 }

```

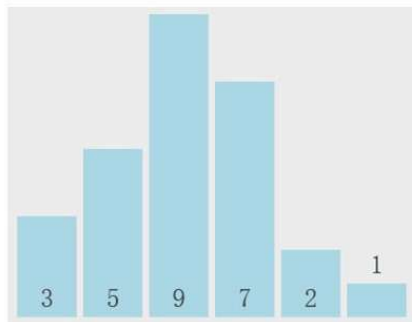


Design and Analysis of Algorithms

Introduction to Bubble Sort

What is Bubble Sort?

- **Definition:**
 - Bubble Sort is a basic comparison-based sorting algorithm.
 - It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
- **Key Idea:**
 - Larger elements "bubble up" to the end of the list in each pass. ↗



How Bubble Sort Works(1)

0	1	2	3	4
15	16	6	8	5

1. Start with an unsorted list: [15, 16, 6, 8, 5]
2. Compare adjacent elements and swap if needed.
3. Repeat until the list is sorted.

How Bubble Sort Works(2)

0	1	2	3	4
16	14	5	6	8

```
for(i=0; i<n-1; i++)
{
    for(j=0; j<n-1; j++)
    {
        if(A[j]>A[j+1])
        {
            temp=A[j];
            A[j]=A[j+1];
            A[j+1]=temp;
        }
    }
}
```

```
for(i=0; i<n-1; i++)
{
    for(j=0; j<n-1-i; j++)
    {
        if(A[j]>A[j+1])
        {
            temp=A[j];
            A[j]=A[j+1];
            A[j+1]=temp;
        }
    }
}
```

How Bubble Sort Works(2)

0	1	2	3	4
16	14	5	6	8

4

16	14	5	6	8
14	16	5	6	8
14	5	16	6	8
14	5	6	16	8
14	5	6	8	16

Pass 1

14	5	6	8	16
5	14	6	8	16
5	6	14	8	16
5	6	8	14	16

Pass 2

5	6	8	14	16
5	6	8	14	16
5	6	8	14	16

Pass 3

Pass 4

How Bubble Sort Works(optimize)

```
for(i=0; i<n-1; i++)
{
    for(j=0; j<n-1-i; j++)
    {
        if(A[j]>A[j+1])
        {
            temp=A[j];
            A[j] =A[j+1];
            A[j+1]=temp;
        }
    }
}

for(i=0; i<n-1; i++)
{
    flag=0;
    for(j=0; j<n-1-i; j++)
    {
        if(A[j]>A[j+1])
        {
            temp=A[j];
            A[j] =A[j+1];
            A[j+1]=temp;
            flag=1;
        }
    }
    if(flag == 0)
        break;
}
```

```
If x<0
    y=x
    x=x+1
```

```
While i≤100
    s←s-l
    i ←i+1
```

Design and Analysis of Algorithms

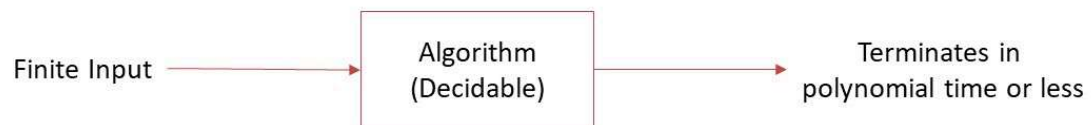
Decidable vs. Undecidable Problems

Topics

- **Decidable Problems**
- **Undecidable Problems**
- **Polynomial vs. Exponential Time**

Decidable Problems

- The problem for which **efficient algorithm** exists.
- Takes **Polynomial time** or less to execute.

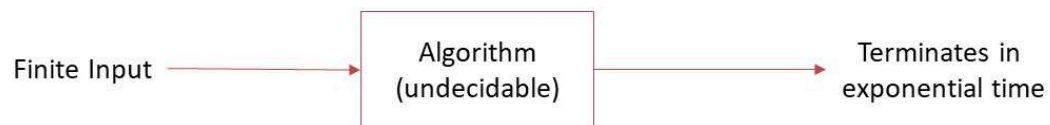


Topics

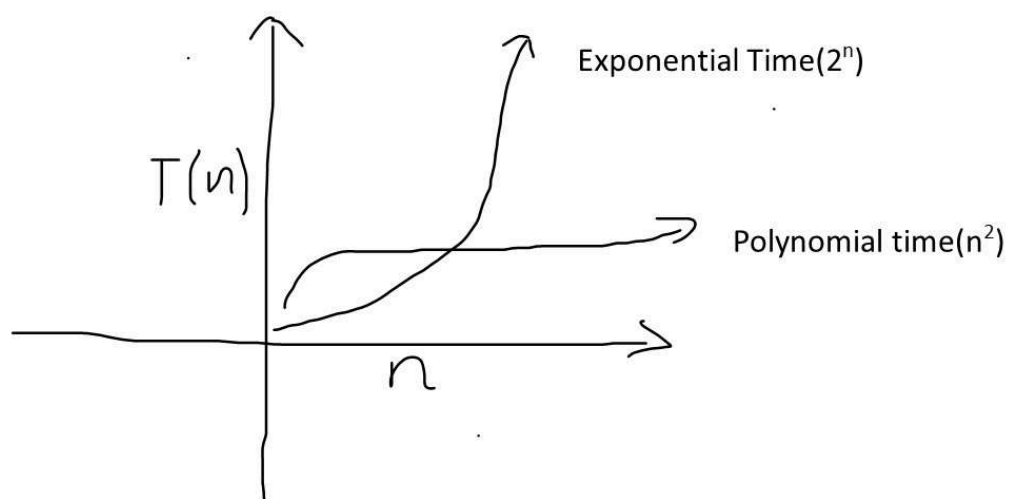
- Decidable Problems
- **Undecidable Problems**
- Polynomial vs. Exponential Time

Undecidable Problems

- The problem for which **no efficient algorithm** exists.
- Takes **exponential time** to execute.

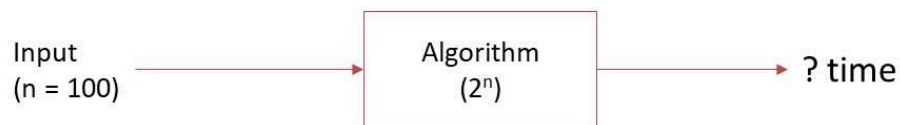


Polynomial vs. Exponential time



The Nature of Undecidable Problems

Let's say that the input size is 100.



The number of instructions = 2^{100}

The Nature of Undecidable Problems

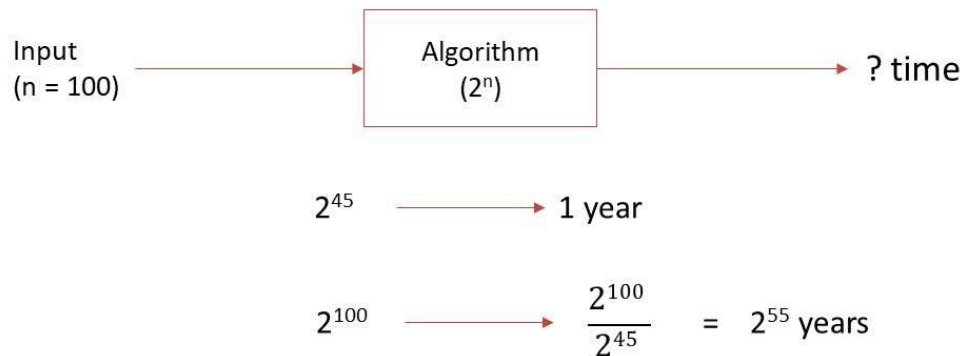
Consider the world's fastest computer capable of executing 2^{20} instructions per second.

In 1 year, the computer can execute.

$$\begin{aligned} & 2^{20} * 60 * 60 * 24 * 365 \\ & \approx 220 * 2^6 * 2^6 * 2^5 * 2^8 \\ & \approx 2^{45} \text{ instructions} \end{aligned}$$

The Nature of Undecidable Problems

Let's say that the input size is 100.



单选题 1分

(1) The definiteness property of an algorithm states that

- ☐ A An algorithm should take less time and memory.
- ☐ B An algorithm must terminate in finite time.
- ☐ C An algorithm must be clear and unambiguous.
- ☐ D An algorithm must produce at least one output.

单选题 1分

(2) An algorithm is an ____ concept, whereas a program is a ____ implementation.

- ☐ A Concrete, abstract
- ☐ B Abstract, concrete
- ☐ C Difficult, simple
- ☐ D Simple, difficult

单选题 1分

(3) Is it true that an algorithm depends upon the size of the RAM?

- ☐ A Yes
- ☐ B No

单选题 1分

(4) Assume that the time to execute 1 operation by a computer is 10ms. How much time the computer will take to execute 3^n operations where $n = 5$?

- ☐ A 243 ms
- ☐ B 243 ms
- ☐ C 9 seconds
- ☐ D 2.43 seconds

单选题 1分

(5) A decidable problem is a problem for which ____ exists.

- ☐ A No efficient algorithm
- ☐ B Efficient algorithm
- ☐ C No algorithm
- ☐ D An algorithm

单选题 1分

(6) An undecidable problem terminates in

- ☐ A Polynomial time.
- ☐ B Logarithmic time.
- ☐ C Constant time.
- ☐ D Exponential time.

单选题 1分

(7) How many operations does the following statement have?

$$x = y * z - 3 / p + 5$$

- ☐ A 5
- ☐ B 7
- ☐ C 4
- ☐ D 11

单选题 1分

(8) Which of the following are the characteristics of an algorithm?

- ☐ A Finiteness, ambiguity, and speed.
- ☐ B Finiteness, definiteness, and effectiveness.
- ☐ C Definiteness, ten inputs, and no output.
- ☐ D None of the above.

单选题 1分

⚙️ 设置

此题未设置答案，请点击右侧设置按钮

此处添加题目描述

- ☐ A 此处添加选项内容
- ☐ B 此处添加选项内容
- ☐ C 此处添加选项内容
- ☐ D 此处添加选项内容

提交

